	Components.
11	`useImperativeHandle`—Allows customizing refs for controlled interactions between components. Relevant for advanced topics.
12	`useDebugValue`—Helpful for debugging custom hooks but rarely asked in interviews.
13	`useDeferredValue` and `useTransition`—Related to concurrent rendering and performance optimization. Typically asked in advanced interviews or senior-level roles.

Finally, low-priority hooks in table A.9 cater to niche use cases and are rarely emphasized in interviews unless the role explicitly requires them.

Table A.9 Low-priority Hooks React Hooks for frontend interview preparation

Priority	Hooks
14	`useSyncExternalStore`—Used for integrating external stores like Redux or MobX. Relevant if the job involves external state management libraries.
15	`useId`, `useInsertionEffect`, and `useOptimistic`—Specialized hooks with niche applications. Prioritize them only if relevant to the role of project requirements.

A.4.2 Custom Hooks: best practices

Custom Hooks are reusable functions that encapsulate component logic, allowing developers to share stateful logic across multiple components. They follow the rules of React hooks and are named with the prefix "use" (e.g., `useFetch`, `useLocalStorage`).

Custom Hooks encapsulate complex component logic, promoting code reusability, enhancing readability, and simplifying maintenance. By abstracting away intricate implementation details, custom Hooks make components cleaner, more focused, and easier to understand.